

Docker Compose 简答题话术模板

面试专用：结构化表达 + 命令 + 场景，直接背诵

模板一：docker compose down / stop / rm 区别

标准话术（30秒版本）

"这三个命令的核心区别是**清理力度**不同：

`docker compose down` 是**彻底清理**，停止并删除容器、网络，还可以加 `-v` 删除卷，适合环境重置；

`docker compose stop` 是**临时停止**，只停容器但保留状态，后面可以用 `start` 恢复，适合临时维护；

`docker compose rm` 在 compose 里很少用，它是删除已停止的容器，通常直接用 `down` 更彻底。"

详细版（1分钟版本）

命令	停止容器	删除容器	删除网络
<code>docker compose down</code>	✓	✓	✓
<code>docker compose stop</code>	✓	✗	✗
<code>docker compose rm</code>	✗	✓	✗

生产场景举例： - 开发环境调试： `docker compose stop` → 改代码 → `docker compose start` - 部署新版本： `docker compose down` → 拉新镜像 → `docker compose up -d` - 清理磁盘： `docker compose down -v --rm all` → 容器网络卷镜像全删

加分点： - `down` 可以加 `--remove-orphans` 删除 compose 中已移除的服务 - `stop` 是发 SIGTERM, graceful stop; `kill` 才是 SIGKILL

模板二：容器间网络不通排查

标准话术（30秒版本）

"我的排查思路是**由外到内、分层定位**：

第一步，看服务状态， `docker compose ps` 确认两个容器都在运行；

第二步，看日志， `docker compose logs web` 看是连接超时还是拒绝；

第三步，进容器测连通， `docker compose exec web sh`，然后 `ping db` 和 `nslookup db` 测试 DNS 解析；

第四步，检查网络配置， `docker network inspect` 看两个容器是否在同一网络；

第五步，检查 db 端口监听， `docker compose exec db ss -tlnp` 看 MySQL 是否真的在监听。"

详细版（1分钟版本 + 命令）

第一层：确认服务状态

```
docker compose ps
# 看 State 列, 确认 web 和 db 都是 Up, 不是 Restarting 或 Exit
```

第二层：看日志定位错误类型

```
docker compose logs web --tail 50
# 如果是 "Connection refused" → db 没监听端口
# 如果是 "Connection timed out" → 网络不通或防火墙
# 如果是 "Unknown host" → DNS 解析失败
```

第三层：进容器测试 DNS 和连通性

```
docker compose exec web sh

# 测试1：DNS 解析
nslookup db
# 正常应该返回 db 容器的 IP

# 测试2：ICMP 连通
ping db -c 3

# 测试3：端口连通（如果 db 是 MySQL）
nc -zv db 3306
# 或者 telnet db 3306
```

第四层：检查网络配置

```
# 看项目名（默认是目录名）
docker compose config | grep name

# 检查网络
docker network ls | grep <项目名>
docker network inspect <项目名>_backend
# 看 Containers 字段里有没有 web 和 db
```

第五层：检查目标服务端口

```
docker compose exec db sh
ss -tlnp | grep 3306
# 或者 netstat -tlnp | grep 3306
# 确认 MySQL 真的在监听 0.0.0.0:3306, 不是 127.0.0.1:3306
```

第六层：检查环境变量

```
docker compose exec web sh
env | grep DB
# 确认 DB_HOST 是 "db"（服务名），不是 IP 地址或 localhost
```

加分点： - 如果用了 `depends_on` 但 db 还在初始化，web 连不上是正常的，要加 `healthcheck` + `condition: service_healthy` - 如果网络不通但 DNS 正常，检查 iptables: `iptables -L -n | grep <项目名>`

模板三：多文件管理配置（docker-compose.yml + docker-compose.prod.yml）

标准话术（30秒版本）

"我用多文件管理是为了**环境隔离和配置复用**：

`docker-compose.yml` 放基础配置，所有环境共用；

`docker-compose.prod.yml` 放生产特有的覆盖，比如资源限制、日志策略、健康检查；

启动时用 `-f` 按顺序加载，后面的覆盖前面的，比如 `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`。"

详细版（1分钟版本）

为什么不用一个文件？

```
# 错误做法：一个文件里写满条件判断
services:
  web:
    ports:
      - "80:80"
  # 生产要限制资源，开发不要，怎么写？写两个 ports？yaml 不支持条件判断
```

正确做法：文件拆分

```
.
├── docker-compose.yml           # 基础配置（所有环境共用）
├── docker-compose.override.yml  # 开发覆盖（自动加载，不用 -f）
├── docker-compose.prod.yml     # 生产覆盖
└── docker-compose.test.yml     # 测试覆盖
```

docker-compose.yml（基础）

```
services:
  web:
    image: nginx:alpine
    ports:
      - "80:80"
```

```
db:
  image: mysql:8.0
  environment:
    MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
```

docker-compose.prod.yml (生产覆盖)

```
services:
  web:
    restart: always
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      retries: 3
  db:
    restart: always
    volumes:
      - db-data:/var/lib/mysql
```

docker-compose.override.yml (开发覆盖, 自动加载)

```
services:
  web:
    volumes:
      - ./html:/usr/share/nginx/html:ro    # 代码热更新
    ports:
      - "8080:80"                          # 开发用8080, 避免权限问题
```

加载规则:

```
# 开发环境 (自动加载 override.yml)
docker compose up -d
# 实际加载: docker-compose.yml + docker-compose.override.yml

# 生产环境 (手动指定)
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

```
# 实际加载：基础 + 生产覆盖，后面的覆盖前面的

# 测试环境
docker compose -f docker-compose.yml -f docker-compose.test.yml up -d
```

加分点： - `.env` 文件配合多文件使用，不同环境放不同的 `.env` - CI/CD 中，生产部署脚本里写死 `-f` 顺序，避免误加载 `override - docker compose config` 可以查看合并后的最终配置，验证是否正确

模板四：bind mount vs named volume 选型

标准话术（30秒版本）

"bind mount 和 named volume 都是数据持久化方式，但适用场景不同：

bind mount 是直接把宿主机路径映射到容器，适合开发环境代码热更新，但路径依赖宿主机，权限容易出问题；

named volume 由 Docker 管理存储位置，适合生产环境数据持久化，可以跨容器共享，备份也方便。"

详细版（1分钟版本）

维度	bind mount	named volume
语法	<code>./host:/container</code>	<code>vol-name:/container</code>
存储位置	宿主机指定路径	<code>/var/lib/docker/volumes/</code>
预先创建	路径必须存在	自动创建
权限问题	UID/GID 容易冲突	Docker 自动处理
跨主机迁移	依赖路径一致性	可导出/导入
备份方式	直接 <code>tar</code> 宿主机目录	<code>docker volume backup</code>
适用场景	开发代码热更新、配置文件	生产数据库、日志、缓存

生产建议：

```

services:
  web:
    # 开发用 bind mount, 生产用 named volume
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro # 配置文件用 bind mount (版本控制)
      - web-data:/var/log/nginx # 日志用 named volume
  db:
    volumes:
      - db-data:/var/lib/mysql # 数据库必须用 named volume

```

加分点： - tmpfs mount 是第三种方式，数据存内存，容器停止即丢失，适合敏感缓存 - 生产环境 bind mount 的 `:ro` 只读模式很重要，防止容器误改宿主机文件

模板五：depends_on 的坑与解决方案

标准话术（30秒版本）

" `depends_on` 只控制启动顺序，不保证服务就绪。比如 db 容器启动了，但 MySQL 还在初始化，web 去连接就会报错。

解决方案有两种：一是 compose 的 `healthcheck` + `condition: service_healthy`，二是应用层自己做重试和超时。"

详细版（1分钟版本）

问题复现：

```

services:
  web:
    depends_on:
      - db
  db:
    image: mysql:8.0
# 实际：db 容器启动后，MySQL 初始化需要 10-30 秒，web 立即启动会连接失败

```

方案1：compose healthcheck（推荐）

```

services:
  web:
    depends_on:

```

```

    db:
      condition: service_healthy # 等 db 健康后才启动
db:
  image: mysql:8.0
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_PASSWORD}"]
    interval: 5s # 每5秒检查一次
    timeout: 3s # 单次检查超时3秒
    retries: 10 # 失败10次才标记不健康
    start_period: 30s # 启动后30秒内不计算失败次数

```

方案2：应用层重试（更可靠）

```

# Python 示例：启动时循环重试连接
import time
import mysql.connector

for i in range(30):
    try:
        conn = mysql.connector.connect(host='db', user='root', password='xxx')
        break
    except Exception:
        time.sleep(2)

```

加分点： - `condition: service_healthy` 需要 compose v2.1+ 或 v3.4+（文件顶部加 `version: '3.8'`） - 有些团队不用 compose 的 healthcheck，全靠应用层重试，因为更灵活 - K8s 里用 readiness probe 解决同样问题，思路一样

模板六：compose 项目名与容器命名规则

标准话术（30秒版本）

"compose 默认用**所在目录名**作为项目名，容器名格式是 `项目名_服务名_序号`，比如 `myapp_web_1`。

可以通过 `COMPOSE_PROJECT_NAME` 环境变量或 `-p` 参数自定义，避免目录名冲突。"

详细版（1分钟版本）

```
# 目录结构
~/projects/myapp/
├── docker-compose.yml

# 默认项目名 = myapp
# 容器名：myapp_web_1, myapp_db_1
# 网络名：myapp_default
# 卷名：myapp_db-data
```

自定义项目名：

```
# 方式1：命令行
docker compose -p myapp-prod up -d

# 方式2：环境变量
export COMPOSE_PROJECT_NAME=myapp-prod
docker compose up -d

# 方式3：.env 文件（推荐）
echo "COMPOSE_PROJECT_NAME=myapp-prod" > .env
```

为什么要改项目名？ - 同一台机器上跑同一套系统的多个实例（开发/测试/生产） - 避免目录名带特殊字符导致容器名不合法

加分点： - `container_name` 可以固定容器名，但 `scale` 多个实例时会冲突 - 网络名默认是 `项目名_default`，自定义网络名是 `项目名_网络名`

模板七：生产环境 compose 最佳实践

标准话术（1分钟版本）

"我在生产环境用 compose 会注意这几点：

第一，**安全**：不用 `privileged`，容器以非 `root` 运行，敏感数据用环境变量或 `secrets`；

第二，**稳定**：配置 `restart: unless-stopped`，加 `healthcheck` 自动恢复，日志做轮转防止磁盘满；

第三，**资源**：限制 CPU 和内存，防止单个容器拖垮宿主机；

第四，**网络**：自定义网络隔离，内部服务用 `internal: true` 禁止外网访问；

第五，**数据**：数据库用 named volume，定期备份，compose 文件和配置进版本控制。"

详细版（配置示例）

```
version: '3.8'

services:
  web:
    image: nginx:alpine
    restart: unless-stopped
    # 安全: 非 root 运行
    user: "1000:1000"
    # 资源限制
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 256M
        reservations:
          cpus: '0.25'
          memory: 128M
    # 健康检查
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    # 日志轮转
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    networks:
      - frontend

  db:
    image: mysql:8.0
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password # 用 secrets
    secrets:
```

```
- db_password
volumes:
  - db-data:/var/lib/mysql
networks:
  - backend
# 内部网络，不连外网

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true

volumes:
  db-data:
    driver: local

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

背诵建议

场景	背哪个模板
被问到 compose 命令区别	模板一
被问到网络排障	模板二
被问到环境管理	模板三
被问到存储选型	模板四
被问到服务依赖	模板五
被问到命名规则	模板六
被问到生产实践	模板七

练习方法： 1. 先读一遍，理解每层的逻辑 2. 遮住答案，用自己的话复述 3. 计时：每个模板 30 秒内说完 4. 结合实际命令，不要只背概念

打印出来，面试前每天过一遍，直到能脱稿流利表达。